

TRACI: Network Acceleration of Input-Dynamic Communication for Large-Scale Deep Learning Recommendation Model

Guyue Huang
University of California, Santa
Barbara
Santa Clara, USA
guyue@ucsb.edu

Hao Li
University of Minnesota
Minneapolis, USA
li003703@umn.edu

Le Qin
Hong Kong University of Science and
Technology (Guangzhou)
Guangzhou, China
lqin674@connect.hkust-gz.edu.cn

Jiayi Huang
Hong Kong University of Science and
Technology (Guangzhou)
Guangzhou, China
hjiy@hkust-gz.edu.cn

Yangwook Kang
Samsung Electronics
San Jose, USA
yangwook.k@samsung.com

Yufei Ding
University of California, San Diego
San Diego, USA
yufeid@ucsd.edu

Yuan Xie
Hong Kong University of Science and
Technology
Hong Kong, China
yuanxie@ust.hk

Abstract

Large-scale deep learning recommendation models (DLRMs) rely on embedding layers with terabyte-scale embedding tables, which present significant challenges to memory capacity. In addition, these embedding layers exhibit sparse and random data access patterns, which demand high memory bandwidth. Multi-GPU systems provide a promising solution, allowing for the scaling of both memory and aggregated bandwidth. However, network communication bandwidth becomes a bottleneck for multi-GPU DLRM systems. Overcoming the communication bottleneck is crucial to unlocking the potential of multi-GPU systems for efficient and high-performance DLRM training.

This paper introduces TRACI, an in-network acceleration architecture designed to optimize the communication operator in embedding layers: Aggregation. While in-network acceleration has proven successful for the All-Reduce communication collective, existing solutions do not directly apply to Aggregation due to two key challenges. Firstly, existing multi-GPU shared memory operations are designed for point-to-point communication and do not allow the network to proactively optimize communication. Secondly, in Aggregation, data transfer patterns are dynamic and dependent on input, demanding the network to dynamically discover and exploit message connections on-the-fly. To address these challenges, we propose a solution that involves a novel network transaction and switch hardware design. We introduce a new network transaction that augments messages with input reuse and output reuse identifications, and can empower the network to proactively reduce

data transfers. Additionally, we present an in-switch cache and a reduction table structure, effectively harnessing input and output reuse opportunities. Using these innovations, TRACI achieves up to 4.04 $\times$ and an average of 3.12 $\times$ speedup to Aggregation for a 64-GPU system.

CCS Concepts

• **Networks** $\rightarrow$ **Network architectures**; • **Computer systems organization** $\rightarrow$ **Distributed architectures**.

Keywords

Deep Learning Recommendation System, GPU

ACM Reference Format:

Guyue Huang, Hao Li, Le Qin, Jiayi Huang, Yangwook Kang, Yufei Ding, and Yuan Xie. 2025. TRACI: Network Acceleration of Input-Dynamic Communication for Large-Scale Deep Learning Recommendation Model. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*, June 21–25, 2025, Tokyo, Japan. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3695053.3731105>

1 Introduction

Deep learning recommendation systems (DLRMs) are widely applied in web applications to rank and personalize ads [7, 8, 11, 13, 14, 40, 44]. Large-scale DLRMs consume a significant fraction of datacenter cycles [13, 18, 19, 34] and thus are very important to optimize.

The key components inside DLRM are the embedding layers, which use learned embedding tables [35] to encode customers and product items. To efficiently run embedding layers, we need both large memory capacity and high memory bandwidth, due to the terabyte (TB) scale model size [1, 33] and the random-access pattern of embedding lookup. Although CPU-only systems [2, 16, 43] and CPU-GPU heterogeneous systems [4, 12, 25] have been proposed in



This work is licensed under a Creative Commons Attribution 4.0 International License. *ISCA '25, Tokyo, Japan*

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1261-6/25/06
<https://doi.org/10.1145/3695053.3731105>

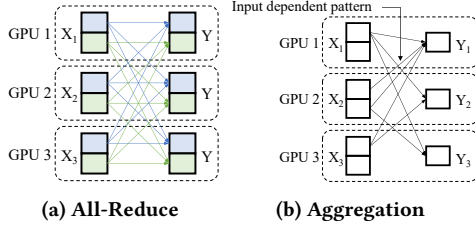


Figure 1: Comparison of communication pattern in All-Reduce and Aggregation.

the past, currently the most promising way to meet the demand for both memory capacity and memory bandwidth is using multi-GPU systems [2, 33, 36, 39]. Unfortunately, multi-GPU DLRM systems are challenged by excessive cross-GPU communication that dominates execution time (Section 2.2).

Overcoming the communication bottleneck is crucial to unleashing the potentials of multi-GPU systems for DLRM. The communication operator in the embedding layer is Aggregation, which gathers embedding entries from the embedding table and accumulates them in the output features, as shown in Figure 1b. Because the embedding table and the output features are partitioned among GPUs, the data access in Aggregation involves a significant amount of cross-GPU network communication.

To accelerate Aggregation, our key insight is the presence of input reuse and output reuse. On the one hand, if the same input vector is added to many output vectors on other GPUs, input reuse means we should avoid repeated injecting the same input to the network to reduce traffic. On the other hand, if provided that many input vectors will be reduced to the same output vector, output reuse means we should reduce them ahead of time so that only the reduction result needs to be transmitted to the GPU holding the output, leading to reduced network traffic as well. The two types of reuse present significant opportunities of reducing the network traffic: harnessing either reuse type can theoretically provide larger than 3× bi-sectional traffic reduction in Aggregation, as elaborated in Section 2.3.

However, harnessing both input reuse and output reuse in multi-GPU systems for Aggregation is challenging, and has never been solved in previous approaches (Section 2.4). *The first challenge comes from the limitations of existing cross-GPU memory operations.* Existing memory operations in the multi-GPU shared memory architecture are either load-style or store-style operations that employ point-to-point network transactions. They do not allow the network to proactively reuse input or output, both semantically and at the network transaction level. *The second challenge comes from the input-dependent nature of Aggregation.* To exploit input reuse and output reuse, the system needs to have a good knowledge of the connections between individual network messages. However, in Aggregation, the data transfer pattern is input-dependent and always changing. We need to design a mechanism to dynamically discover and exploit the two types of reuse.

To tackle these challenges, we introduce TRACI, the first in-network acceleration architecture for Aggregation. TRACI is built on three key methods. **Firstly**, we propose a new multi-GPU shared

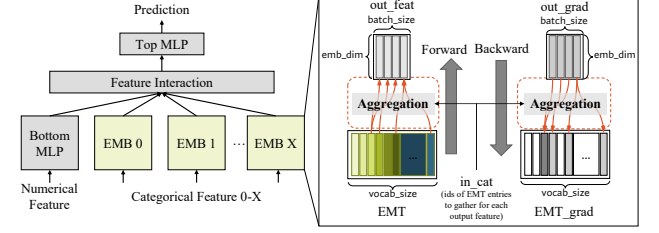


Figure 2: The architecture of DLRM and the operation performed in an embedding layer.

memory operation called *GetReduce*, as the software-hardware interface that semantically allows in-network acceleration. In addition, the *GetReduce* network transaction makes a network message carry both the input address and the reduction output address of a data transfer, which is essential for the network to dynamically discover input and output reuse between messages. **Secondly**, we propose an in-switch caching mechanism to harness input reuse dynamically. The key insight is to cache data from messages that previously travel through a network switch, and make the switch directly respond upon a cache hit, to avoid repeatedly injecting the same input to the network. **Thirdly**, we propose an in-network reduction mechanism to harness output reuse dynamically. On top of the existing in-network reduction methods that leverage a reduction table, our key enhancement is to use a counter mechanism that dynamically increments when seeing a request and decrements when receiving a response. This helps us tackle the dynamic number of messages to reduce.

We comprehensively evaluate the performance of TRACI. In a 64-GPU system, TRACI can bring an average of 3.12× speedup to Aggregation in asynchronous recommendation model processing, and average 2.05× and 2.29× speedup in synchronized processing with small and large batch size respectively. Ablation studies show that both reduction and caching contribute significantly to the final speedup, and combining them is necessary for our system to adapt to different system scales and dataset characteristics.

This paper makes the following contributions:

- Motivated by the input reuse and output reuse opportunities in Aggregation, we present a new multi-GPU shared memory operation, *GetReduce*, which empowers the network to perform input and output reuse optimizations.
- We propose in-switch cache and in-network reduction, the mechanisms to harness input and output reuse in an input-dynamic communication operation like Aggregation.
- We perform extensive evaluations for TRACI and demonstrate significant speedup on a variety of datasets.

2 Background

2.1 DLRM Model Architecture and Aggregation

The architecture of the DLRM [35] is shown in Figure 2, consisting of the bottom-MLP layer, embedding layers, feature interaction and the top-MLP layer. The key component of the DLRM network is the embedding layers (EMBs). The embedding layer transforms a categorical input feature, such as user ID(s) and item ID(s), into

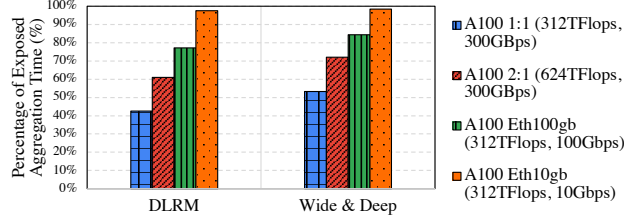


Figure 3: Percentage of end-to-end time spent on exposed communication of EMB in DLRM training on 128 A100 GPUs (16 node, 8 GPU/node). The legend shows (computation throughput, network bandwidth) per GPU. In four configurations, the ratio of the network bandwidth and computation throughput decreases.

an embedding vector. Each embedding layer contains an embedding table (EMT). Define the dimension of embedding feature as `emb_dim`, and the number of embeddings in the table as vocabulary size (`vocab_size`), an EMT is a matrix of size `[vocab_size, emb_dim]`. Define the output of embedding operation as matrix `out_feat`, the number of input samples as `batch_size`, `out_feat` is a matrix of size `[batch_size, emb_dim]`. EMB is an Aggregation operation, where the EMT is input and `out_feat` is output. The input categorical features to this layer, defined as `in_cat`, contain the data transfer pattern, i.e. which embedding entries are accumulated to each output feature. The backward of EMB is another Aggregation, whose input is the gradients of the output feature (defined as `out_grad`, the same shape as `out_feat`), and the output is the gradients of EMT (defined as `EMT_grad`) with the same shape as EMT. The data access pattern is the inverse of that in the forward, and can be calculated from `in_cat`.

2.2 Communication Bottleneck in DLRM Systems

The terabyte scale size of the embedding tables (EMTs) in DLRM models exceeds the memory capacity of a single GPU. Therefore, state-of-the-art DLRM systems such as the TorchRec [33] need to partition EMTs to multiple GPUs. Meanwhile, the input features are also distributed among all GPUs. This means that a significant amount of EMT accesses in Aggregation, shown in Figure 2, happen across GPUs. Figure 3 shows the fraction of the Aggregation time in the end-to-end DLRM training. The evaluation is performed on ASTRA-SIM [38], a distributed training simulation framework. The four different settings in Figure 3 demonstrate different ratios of computation FLOPS to the network bandwidth. The results show that in DLRM systems, the cross-GPU communication for Aggregation is a dominating factor of the end-to-end execution time, and as the bandwidth-to-computation ratio decreases, Aggregation becomes more of a bottleneck.

2.3 Input Reuse and output reuse in Aggregation

In Aggregation, the input reuse and output reuse are the two types of data reuse among data transfers that can be leveraged to reduce the inter-GPU network traffic. As Figure 4a indicates, the *input*

Table 1: Reuse factor and the theoretical reduction factor of bi-sectional traffic reduction by leveraging reuse.

	Facebook Synthetic	LastFM (Web-Review)	Terabyte (CTR)
Avg. output reuse degree	15.8	95.4	1
Avg. input reuse degree	6.9	94.7	100.3
Bi-sec traffic reduction from output reuse	1.27	3.16	1
Bi-sec traffic reduction from input reuse	1.10	3.14	3.26

refers to the array being read, which is the EMT in DLRM. The *output* refers to the array storing the reduction results, which is the output feature (`out_feat` in Figure 2). As shown in Figure 4b, input reuse means that a vector in the input is reduced to multiple vectors in the output. Input reuse can be exploited as follows: instead of repeatedly sending identical copies of the same data, we push only one copy of the data into the network, and make the network multicast the data autonomously and efficiently. As a symmetric idea, output reuse means that a vector in the output is the reduction of multiple vectors in the input. The way to exploit output reuse is that, since only the reduction result matters, the network can proactively reduce the data and only transmit the reduction result. Combining the two types of reuse, we can reduce the messages pushed into and popped from the network, thus effectively accelerating the Aggregation operation.

Input reuse and output reuse opportunities are both significant in real applications. Table 1 shows the theoretical reduction of bi-sectional traffic if we leverage input and output reuse. We assume a system of 256 GPUs connected in a 2-tapered fat tree topology, and assume the accessed entries and samples are randomly distributed among all GPUs. The input/output reuse degree refers to the number of data transfers that share an input/output. We can observe that exploiting output and input reuse can each provide up to 3.16× and 3.26× speedup.

2.4 Constraints of Previous DLRM Acceleration Methods

Systems that exploit data reuse but fail to leverage input and output reuse simultaneously: Many prior papers proposed hardware or system designs to harness the input or output reuse. The major difference between this thread of work and our paper is that, prior work only exploits *one type of reuse*, which, as we elaborate below, is due to the limitation of optimizing Aggregation *inside GPUs*. To exploit output reuse, prior work proposes to reduce different input vectors in one GPU that share the same one output vector, and then only send the reduction result to the network [21, 23, 24]. To exploit input reuse, prior work proposes to pull an input vector only once from the network, and then add it to all output vectors in the same GPU that need this input [20]. The issue is they cannot be easily composed together, because output reuse should be exploited before network transmission and input reuse should be exploited after transmission. Together they become conflicting: if an input is reduced with other inputs for output reuse, the original input is no longer sent to the network, and thus no input reuse cannot be exploited for it. Our key insight is that by leveraging the input and output reuse *inside the network* instead of before or after transmission, two types of reuse can be simultaneously exploited.

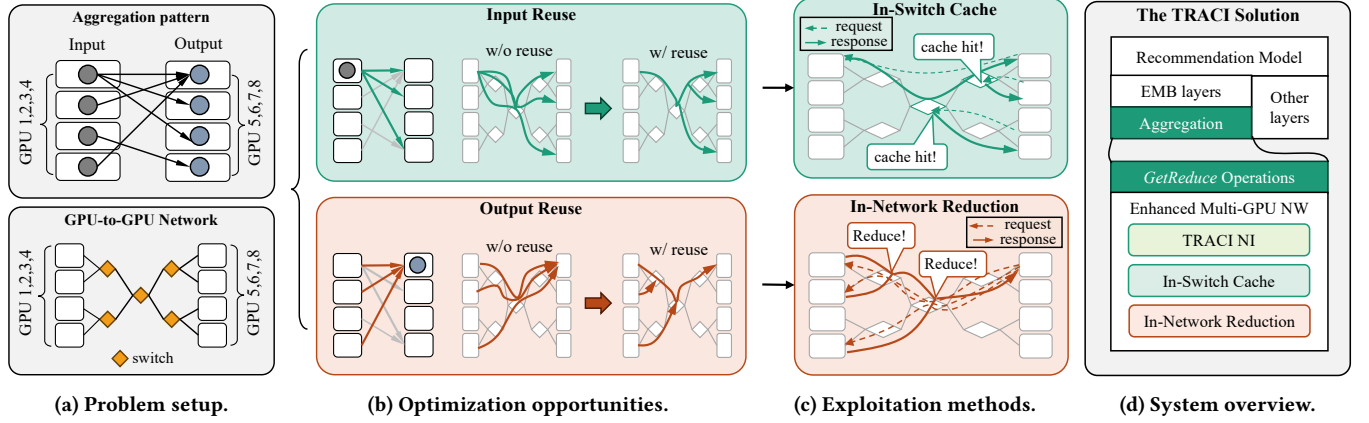
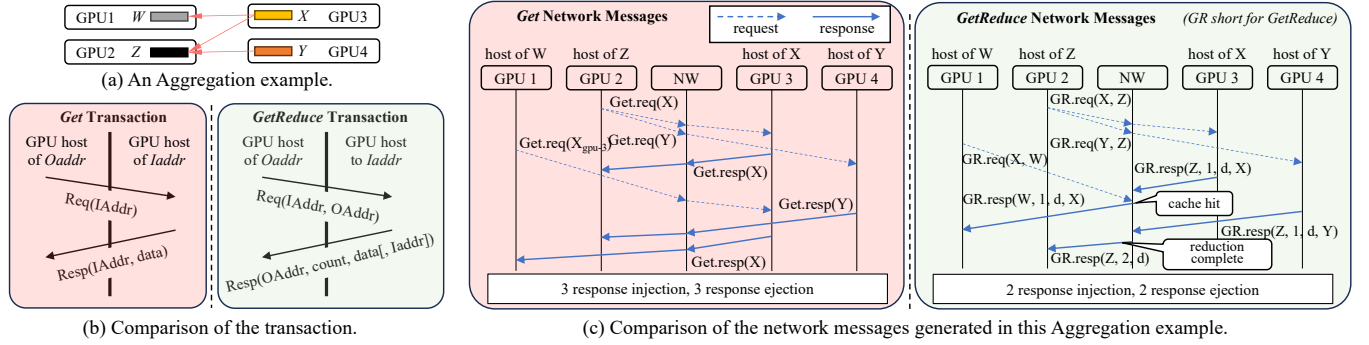


Figure 4: A walk-through of the key opportunities and the main techniques in TRACI system.

Figure 5: Illustration of the *GetReduce* transaction and how it facilitates in-network caching and reduction.

Systems that focus on optimizations orthogonal to data reuse: The *EMT placement problem* considered by RecShard [39] aims to reduce the traffic in a “scaled-up” system composed of multiple levels of heterogeneous memory, in contrast to the “scaled-out” multi-GPU system we study. Due to different assumptions on the architecture, the placement algorithm in RecShard to reduce slow-memory access traffic cannot be applied to multi-GPU systems to reduce the network traffic. The *EMT partitioning problem* considered by TorchRec [32, 33] reveals different methods to partition EMTs. Despite that column-wise and table-wise partitioning incur less network communication than the common row-wise partitioning, none of them can scale to the systems with hundreds of GPUs like the row-wise can due to various limitations: Typically EMT number of rows range from 10^4 to 10^7 , which is orders of magnitude larger than number of tables ($1 \sim 10^3$, refer to Table 3), or number of columns (64 1024), therefore row partitioning is the most scalable and can work for all kinds of models. Thus, the communication bottleneck caused by row-wise partitioning remains an inevitable issue. In summary, none of these system techniques solves the communication challenge in Aggregation.

2.5 The Baseline Architecture

The baseline architecture of this work is a multi-accelerator system with memory-semantic fabrics between accelerators. Example products of such systems include the NVIDIA NVLink Switch System [37], which connects 256 GPUs with NVLinks and NVSwitches into a fat-tree topology, and the TPU-v4 supercomputer [18] with connects 64 TPUs per node and 64 nodes into a 3D torus topology. We mainly use NVIDIA GPU system to illustrate design ideas in the rest of the paper. Unlike traditional network links (e.g., Ethernet), memory-semantic links like NVLink connect all GPUs’ memory into a shared memory space, allowing them to access the memory of another GPU via load/store-like operations. This leads to simple programming model and lower latency for cross-GPU data transfers. In the baseline architecture, the Aggregation can be implemented with the *Get* operation that triggers *Get*-transaction in the GPU-to-GPU network, as shown in Figure 5: GPUs host to the output data will send a request to read input data from another GPU, carrying the *IAddr*, a unique address in the shared memory space. The GPU host to *IAddr* receives this request and generate a response with data in this address.

3 Main Ideas

Section 2.3 reveals that in-network acceleration is a promising approach to leverage input and output reuse simultaneously. However, to realize this idea, we face two challenges.

The first challenge is the limitation of existing shared memory operations. Current shared memory operations like *Get* are point-to-point communication and disallow reuse semantically and at the network transaction level. Specifically, the idea to reduce multiple inputs to one output cannot be implemented with *Get*, because it disallows changing the transmitted data. Also, the network transaction requires one response to every request, so reducing multiple responses to one will violate the protocol and cause failure. In short, existing memory operations cannot enable input and output reuse optimizations.

To tackle this challenge, we propose a new memory operation and corresponding network transactions called *GetReduce*. *GetReduce* messages carry both an input address and an output address, and the network transaction allows one response to reply multiple requests by providing the reduced result. The key to ensure correctness is that the response message carries a counter that indicates how many responses have been reduced, and this can be used to remove waiting requests in the GPU. The details about *GetReduce* are introduced in Section 4.

The second challenge comes from the input-dependent nature of Aggregation. Harnessing input reuse and output reuse requires the system to have a good knowledge of the reuse connections among data transfers. Previously, Klenk *et al.* [22] designed an architecture to exploit output reuse by in-network reduction, but their method can only tackle All-Reduce which has a static data transfer pattern. In both DLRM training and inference, the Aggregation pattern depends on the input dataset, thus the network has to discover and exploit reuse on-the-fly.

To tackle this challenge, we propose a novel switch architecture that includes an in-switch cache and a reduction table. The methods are shown in Figure 4c. The exploitation of input reuse is through caching: when a response travels through a switch, the in-switch cache records the input address and caches the data. Next time when a request travels through the switch, the switch can check the requested address to see if it hits the cache, and upon hit the switch can generate a response message immediately. The exploitation of output reuse is implemented by as follows: when a request travels through a switch, the switch can increment a counter associated with the output address of this request, and expect to receive a response at the same switch in the future. When the response arrives at the switch, the switch performs *reduction* of it with previous responses, and the counter decrements. When the counter decreases to 0, the final result can be sent back to the requester GPU like a normal response message. We also design a flit processing pipeline to combine the cache and reduction functionalities in a switch.

Changes to the existing systems: Figure 4d shows the overview of the TRACI solution and how the current system needs to be modified. The only change in software is to re-implement the embedding layer by a new type of Aggregation that uses the *GetReduce* memory operation. At the hardware level, a new multi-GPU network architecture supports the *GetReduce* operations. It is based on current shared memory network like NVLink and NVSwitches,

but makes the following enhancements. Firstly, we design a new network interface that can process the new *GetReduce* transaction. Secondly, we propose the hardware implementations for the two reuse exploitation methods illustrated in Figure 4c. The new software implementation for EMB and the network architecture together can help us achieve speedup for Aggregation, therefore accelerating the large-scale DLRMs.

4 GetReduce Primitive for Aggregation

The *GetReduce* memory operation: In order to facilitate our optimization, we propose a new memory operation called *GetReduce*. It takes two addresses in the shared memory space: *IAddr* (input address) and *OAddr* (output address). Its functionality is to load the data in address *IAddr* and accumulate it to the data in *OAddr* atomically. We use this memory operation to construct the Aggregation kernel. GPU threads can issue the *GetReduce* operation, and the *OAddr* must be in the device memory of this GPU. When *IAddr* is on a remote GPU, this memory operation will be forwarded to the NVLink port and generate a network transaction.

Network transaction: When a *GetReduce* operation is handled by NVLink port, it first generates a *request* and sends it to the GPU where *IAddr* resides. The request message carries *IAddr* and *OAddr*. Upon receiving the request, the GPU will read its device memory, and send a *response* message to the requester GPU. The response carries *OAddr*, the data from the address, and an integer counter. The counter indicates how many input entries have been reduced in this message, and is initialized to 1. When the counter is 1, the response is also required to carry the *IAddr* with it. When the counter is > 1 , the *IAddr* field has no definition because this is a reduction result of many original inputs. Upon receiving the *GetReduce* response, the requester will generate an atomic-add operation to *OAddr*. The information carried by the two messages is shown in Figure 5(b).

Network acceleration: We accelerate the transaction of *GetReduce* with in-network cache and in-network reduction. Figure 5(a) gives an example Aggregation pattern, which will be implemented with three *GetReduce* operations. Figure 5(c), in its left side, shows the communication happened in a baseline architecture without in-network acceleration. Here GPU-2 sends two requests to the network that are forwarded to GPU-3 and GPU-4 respectively. GPU-1 sends the third request to the network which is forwarded to GPU-3. Similarly, three response messages with data are sent back to GPU-1 and GPU-2 through the network. This architecture does not exploit the reuse opportunity such as the input reuse of *X* and the output reuse of *Z*.

Figure 5(c), on its right side, shows the communication in our proposed architecture that is capable of in-network caching and in-network reduction. Here the GPU-2 also sends two requests to the network and they are forwarded to GPU-3 and GPU-4 like in the baseline. The difference is, when GPU-3 and GPU-4 each send a response message, the network can discover that they will be reduced to the same output, and the network can proactively reduce them to reduce the traffic to GPU-2. For instance, here the network only pops one response message to GPU-2, with the reduced result of two messages carrying *X* and *Y*. This example demonstrates how in-network reduction can exploit output reuse. Moreover, when

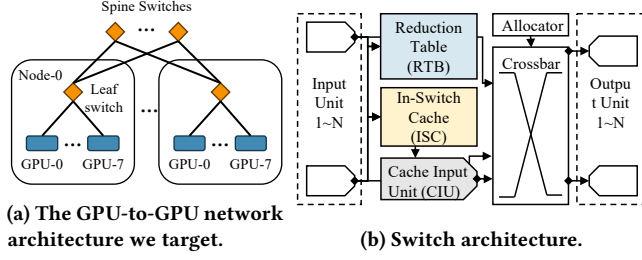


Figure 6: The network and switch architecture.

GPU-1 sends a *GetReduce* message to the network requesting data X , since the network is capable of caching and it has cached the previous message carrying X , the in-network cache can proactively generate a response message, with data X and directly send it back to GPU-1 without notifying GPU-3. This also saves the traffic to and from GPU-3. This example demonstrates how an in-network cache can exploit input reuse.

5 Switch Architecture

This section introduces the hardware architecture of a network switch that supports in-network reduction and in-network cache. We first give an overview of the switch architecture. Next, we introduce two key hardware modules: the reduction table and the in-switch cache. Finally, we introduce how to combine these modules in a flit processing pipeline.

5.1 Overview

Figure 6a shows the structure of the GPU-to-GPU network we target. In the network, GPUs are organized into groups of 8, and each group of GPUs are directly connected to one leaf switch. Every leaf switch is connected to all spine switches, forming a fat tree topology.

Our hardware design is applied to both leaf switches and spine switches. Figure 6b shows the overview of the switch architecture. A baseline switch architecture has the following components: it has N input units and N output units. The input and output units are connected by a crossbar. Every cycle each output unit can only take one flit. Thus an allocator is responsible to select an input unit for each output unit.

Our proposed architecture has the following additional modules: a reduction table (RTB), an in-switch cache (ISC), and a cache input unit (CIU). The functionality of the RTB is in-network reduction: reducing *GetReduce* response packets on-the-fly. All input units are connected to RTB, so that the RTB can process the incoming flits (both *GetReduce* requests and responses). ISC is a cache structure that stores the data carried by previous packets. The ISC can proactively generate a response packet if the *GetReduce* request hits in the cache. Thus, all input units are connected to the ISC, so that ISC can process incoming packets. Because the ISC can generate packets, we design the cache input unit (CIU), which has the identical structure as the regular input units, but its producer is the ISC rather than the links connected to the switch. The CIU is also connected to the crossbar, therefore the allocator now arbitrates

an $N + 1$ -in, N -out network. These are all the modifications to a baseline switch architecture.

5.2 Reduction Table

5.2.1 Functionality of RTB. Figure 7 gives an example that shows how RTB works. Here, GPU-0 and GPU-1 are connected to Switch-0 (SW-0), and GPU-2 is connected to Switch-1 (SW-1). Two switches are directly connected and each has an RTB. Each RTB has multiple entries, and each entry has the following fields as shown in Figure 7: a tag that stores the *OAddr* address, the data field that stores the current reduced result (initialized to 0), a counter that records how many *GetReduce* requests it is waiting for, and another counter that records the number of reduced packets.

Figure 7 gives an example workflow of RTB. GPU-1 sends two *GetReduce* messages to the network with $IAddr = X_{gpu-0}$, and $IAddr = X_{gpu-2}$, respectively. The two requests have identical $OAddr = Y_{gpu-1}$. The address X_{gpu-0} is located on GPU-0, hence SW-0 receives this message and forwards it to GPU-0. The address X_{gpu-2} is located on GPU-2, hence SW-0 receives this message and forwards it to SW-1 and eventually GPU-2 receives the request. Whenever a request is forwarded by a switch, the switch checks the *OAddr* against its RTB. If the address is already in RTB, the waiting counter increments. If the address is not an tag in RTB, the switch tries to allocate a new entry. Hence, after two requests have passed SW-0, SW-0 RTB has an entry tagged with *OAddr* waiting for 2 responses.

In the second snapshot in Figure 7, GPU-0 sends a response carrying data from X_{gpu-0} . When SW-0 receives this packet, it finds the RTB entry associated with the *OAddr* in the packet, Y_{gpu-1} , and decrements the waiting counter as well as incrementing the arrived counter. The SW-0 also performs an addition between the data in the RTB entry and the data carried by the incoming packet. Then, the original response packet, Packet (3), is dropped. Afterward, SW-0 checks if the waiting counter has decreased to 0. In this case, the waiting counter has decreased from 2 to 1, but the entry is still waiting for other responses. Thus, SW-0 does not generate new response packets.

In the third snapshot, GPU-2 sends a response carrying data from X_{gpu-2} . When SW-1 receives the packet and reduces the data to its corresponding RTB entry, SW-1 observes that the waiting counter has decreased to 0. This time, the SW-1 drops the old response, packet(4), and generates a new response, packet (5). Because here there is no reduction happening, the contents of packet (5) is the same as packet (4). When SW-0 receives packet (5), it reduces the data to the current result in RTB, and updates the data in RTB to $d_0 + d_1$. The SW-0 also updates the counters and observes that the waiting counter is now 0. Hence, SW-0 generates a new response. In the response, the *count* field is extracted from the arrived counter, the *data* field is extracted from the RTB entry data, and the *OAddr* field is extracted from the RTB entry tag. Finally, the response (5) carrying the reduction result of two responses is sent to GPU-1.

5.2.2 Design Considerations. Routing: our RTB design assumes that the routing of a *GetReduce* response is the reverse of the corresponding request. If the responses take a different path from the inverse of the request, the RTB in the request path can wait for a response that will never travel through the same switch, which leads to deadlock. Note that the routing between a pair of GPUs

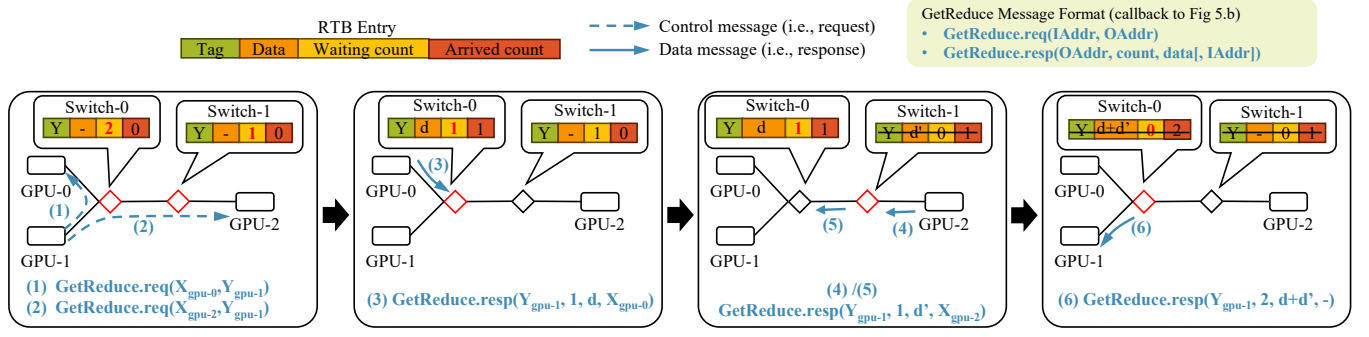


Figure 7: Illustration of how RTB works. The subscripts of addresses X, Y indicate which GPU the address belongs to.

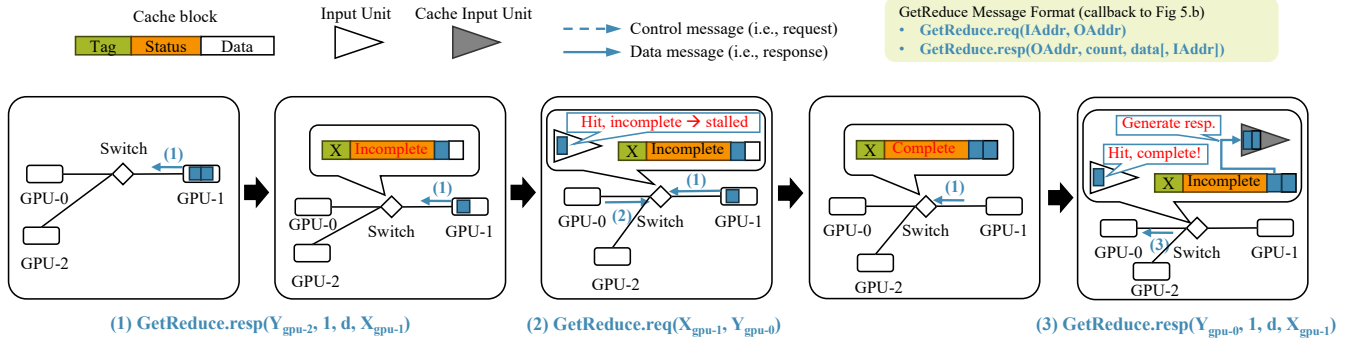


Figure 8: Illustration of how in-switch cache works. The subscripts of addresses X, Y indicate which GPU the address belongs to.

can still be un-deterministic, and we only require the routing of a transaction to be the same. Our method to guarantee it is to hash the $OAddr$ field of the message when we choose among multiple viable routes. As long as the hashing function is random, no obvious workload imbalance will be caused among different routes.

Deadlock prevention: Because the RTB has a finite number of entries, it is possible that RTB fails to allocate a slot for a request because its buffers are all used and waiting for responses. Under such case, we have two choices: to bypass the allocation, and to stall the request until there is an available buffer. Bypassing the allocation implies that the responses cannot be reduced and we lose the opportunity to save network traffic. However, non-selectively blocking the requests can lead to deadlock in the network. Our solution is to classify the messages into two types: for messages that just get injected to the network (i.e., coming from a network interface), we can safely stall them without causing deadlock. For messages that come from another router, we will bypass them under RTB allocation failure to avoid deadlock. If a newly injected message is stalled due to full RTB, we do not add extra buffers to store those requests. Instead, we keep the message in the input port, and this will stop the preceding switch from sending new flits through the link. Compared to always bypassing, the profit of our strategy is to exploit more output reuse and reduce traffic, while the issue is to increase the latency of some transactions. Ultimately, because the Aggregation is mostly bandwidth-bounded rather than latency-bounded, the pros drastically outweigh the cons.

5.3 In-Switch Cache

The in-switch cache (ISC) is a cache structure in the switch with two main functionalities. Firstly, when a *GetReduce* response packet passes this switch, the data can be inserted into the cache. Secondly, when a *GetReduce* request whose $IAddr$ address hits in the cache, the cache can generate a response packet to directly response to the requester GPU.

5.3.1 Functionality of In-Switch Cache. Figure 8 illustrates the functionality of cache insertion. Here, a switch connects GPU-0 and GPU-1. In the first snapshot, GPU-1 sends a response packet carrying data in $IAddr=X_{\text{gpu-1}}$. The data packet contains more than one flit, hence it takes more than one cycle to fully send the packet to the switch. In the second snapshot, the head flit of packet (1) reaches the switch. When the switch observes that the data in address $X_{\text{gpu-1}}$ is not in the cache, the switch allocates a cache block to save the current and future flits of this packet. Because the switch has not received the complete packet, the cache block is now in an *incomplete* status. Note that while the flit is being saved in the cache, it also gets normally forwarded to its next hop.

The third snapshot reflects a situation where an incoming request hits on a cache block that is incomplete. Here, GPU-0 sends a request, packet (2), to the switch, asking for data in address $X_{\text{gpu-1}}$. The switch queries its cache and finds a cache hit on an incomplete cache block. Because the data is not complete, it is not ready to be wrapped as a response. The request packet will be temporally

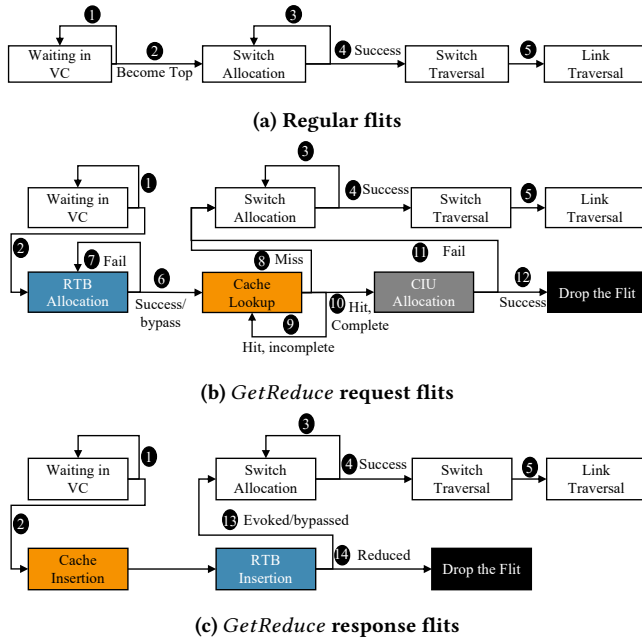


Figure 9: The flit processing pipeline in the switch architecture.

stalled in the same buffer, and will query the cache in the next cycle again.

In the fourth snapshot, the last flit of the packet (1) is transferred to the switch, and the cache block turns from incomplete status to a complete status. Then, in the fifth snapshot, the same request packet that has been stalled in its input unit makes a query again now hits on a complete cache block. Upon this event, the cache tries to allocate a virtual channel in the cache input unit (CIU), and puts a response carrying the data in the cache into the CIU. The response packet will be sent to GPU-0. Meanwhile, the original request packet is dropped.

5.3.2 Design Considerations. Differences from a traditional cache: A major difference of ISC from a classic cache in CPU is that it does not reserve cache blocks for miss memory accesses. A classic cache will reserve a cache-block upon miss, and when the data response is back the data is inserted to the reserved cache-block. If the set is full, eviction also happens when performing reservation. In contrast, ISC does not reserve cache-blocks, and the eviction happens when the response packet enters this switch. This is because our architecture also supports in-network reduction, which implies that a group of previous requests might be responded by one single reduced response. Hence, reservation does not work because this request may not be responded.

Cache coherence handling: It is important to maintain cache coherence for the ISC in order to avoid making GPU access stale data. In prior work that designs in-network caches for key-value stores and graph applications (e.g., Concordia [42]), in order to provide a strong memory consistency model for easy application development, the network caches should employ complex coherence models. We observe that such a coherence model is an overkill

for the multi-GPU network caches that we design. This is because unlike Concordia that are developed for general applications, the current multi-GPU shared memory architecture is dedicated for deep learning training, and uses a weak memory consistency model that are cheaper and more efficient to implement. In this model, the GPU caches rely on explicit global synchronization to flush stale data. We consider having stale data in *network cache* to be acceptable since *GPU caches* can also have stale data. Our method for coherence handling is to invalidate all cache blocks whenever a multi-GPU synchronization happens. For DLRM, adding a multi-GPU synchronization at the end of every training batch is enough for correctness. This mechanism incurs essentially no performance overhead because in training, a multi-GPU synchronization between batches is already required to ensure gradient updates have completed. Invalidating the ISC is done by setting a bit for every entry and we assume this job can be performed within a few cycles.

5.4 Flit Processing Pipeline

In this part, we will introduce the flit processing pipeline in our proposed switch. A flit processing pipeline is the state transition of a flit inside a switch, starting from being inserted in the input unit from an external link, and ending with leaving this switch through another link connected to an output unit. In the baseline switch architecture without cache and RTB, the processing pipeline starts from waiting in an virtual channel (VC), and continues with switch allocation, switch traversal and link traversal, as shown in Figure 9a.

The above introduced pipeline is for flits not belong to the *GetReduce* messages. Figure 9b shows the processing pipeline for the *GetReduce request* flit. The request is a control message and we assume a packet consists of only one flit. At the beginning, the flit waits in the VC of an input unit like a regular flit. It enters the RTB allocation state under the condition ② that it becomes the top of the VC. If RTB already has an entry, or successfully allocates a buffer entry for the *OAddr* address of this packet, then the flit moves on to the cache lookup state (⑥). If the allocation fails but the switch decides to bypass allocation due to deadlock prevention reasons introduced in Section 5.2, the flit is also transferred to cache lookup. Otherwise, the flit stays at the RTB Allocation state, stalled in its VC (⑦). If the cache lookup gives a miss, the flit enters the switch allocation and gets normally transmitted by the crossbar (⑧). If the cache lookup gives a hit on an incomplete cache block, the flit is stalled waiting for the cache block completion (⑨). If it is a hit on a complete cache-block, the flit enters the CIU Allocation state (⑩). In the CIU Allocation state, the CIU tries to allocate a VC for inserting the generated response message. If successful, a response will be inserted in the CIU VC and this request flit gets dropped (⑫). Otherwise if all CIU VCs are occupied, this request cannot be responded here eventually, and it is transferred to switch allocation to for a normal transmission (⑪).

We just introduced the processing pipeline for a request flit. The pipeline for flits in a response packet is shown in Figure 9c. Like other two pipelines, a response flit starts by waiting in the VC of an input unit, and upon becoming the head of the VC, it enters the Cache Insertion state. In this state, the ISC checks the *IAddr* address of this packet against the tag array to determine if the data

Table 2: System and hardware configurations.

Topology	2-tapered fat tree, 8 GPU / leaf switch, 4 spine switches
Link Latency	500ns
Switch Configuration	10 virtual-channel (VC) per channel, 1200 VC depth, round-robin arbitration
Flit size	64 Byte
Link Bandwidth	64 GB/sec

Table 3: Benchmark dataset characteristics.

	Facebook Synthetic	CTR			Web-Review		
	17 datasets	Kaggle Ad	Avazu	Terabyte	Amazon Movies&TV	LastFM	DBLP
#Table	856	26	20	26	1		
Average Table Size (10e6 entries)	2.72	1.19	0.45	1.76	2.64	39.83	23.67
#Lookups	65536	41M	36M	200M	57k	422k	379k
Average Pooling Size	8.4	1			47.5	95.4	63.4

in this packet needs to be cached, and if the tag is absent, the ISC allocates a cache block and copies the data in the flit. After finishing cache insertion, the flit enters the RTB insertion state, when the RTB checks the *OAddr* address of this packet against the RTB tag array, to determine if the data in this packet needs to be reduced. If the packet does not need to be reduced to any buffer (bypassed), or it is the last packet waited by the buffer and the reduced result should be sent back (evoked), the flit enters the switch allocation state and is normally transmitted by the crossbar⁽³⁾. Note that if this is an evoked packet, then the data is the reduction result of the original flit and the data in the RTB buffer. On the other hand, if the data is reduced and it is not the last packet this buffer slot is waiting for, the data is added to the buffer and then the flit is dropped⁽¹⁴⁾.

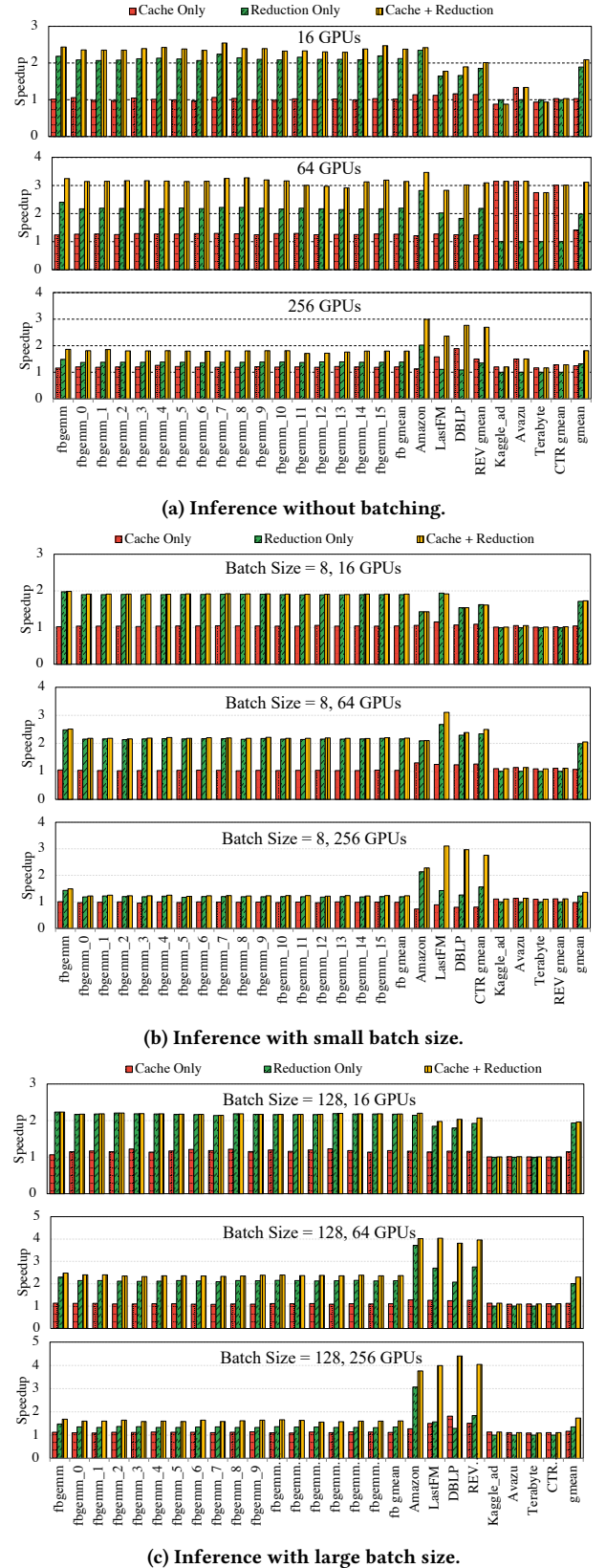
6 Experiments

6.1 Methodology

We implement a cycle-accurate performance simulator for our network architecture and the baseline architecture by extending gem5 Garnet [3] simulator. We use this simulator to evaluate the latency of Aggregation in the embedding layer forward. The network setup in our architecture follows the public datasheet of the NVLink and the Hopper NVlink-Switch system [15, 37], as listed in Table 2. The largest system that we simulate contains 256 GPUs, which is enough to fit all the DLRM embedding tables (EMTs)¹. Employing more GPUs will cause unnecessary embedding communication overhead, so we believe larger systems will simply leverage the data-parallelism, and the speedups will be similar to 256 GPUs.

We evaluate TRACI on 23 DLRM applications detailed in Table 3 (17 facebook DLRM datasets, 3 Click-Through Rate (CTR) applications, and 3 web review applications). In terms of embedding table

¹NVIDIA A100 has a 80GB of DRAM and H100 has 96GB, 256 of them are enough to fit the largest DLRMs with 10 TB EMT [33].

**Figure 10: Speedup of Aggregation in DLRM inference.**

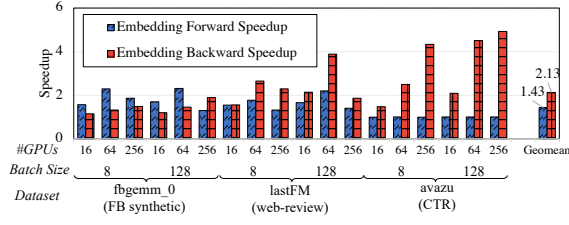


Figure 11: Speedups of Aggregation in DLRM training.

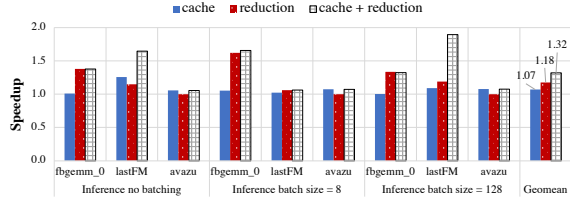


Figure 12: Speedup of Aggregation in DLRM inference on a system consisting of 64 GPUs and uses mesh 3D (4×4×4) topology.

size, the facebook DLRM has around 140G parameters, CTR applications have 0.5-2G parameters, and web review applications have 0.1-2.5G parameters. When evaluating end-to-end time, we assume all applications have the same MLP structure: a 3-layer bottom-MLP with hidden size 512, 256 and 64, and a 5-layer top-MLP with hidden size 1024, 1024, 512, 256, and 1. The Embedding communication speedup in Section 6.2 are evaluated on our cycle-accurate network simulator, whereas the non-embedding layers are evaluated with Astra-Sim, and end-to-end results in Section 6.5 are a combination of those two parts.

6.2 Communication Speedup

To cover a wide range of real-world scenarios, we simulate three different settings: (1) Inference without batching. This operation performs DLRM forward pass on the dataset without synchronization between a fixed number of samples. It represents the scenario of inference with no batching in latency-critical applications. (2) Inference with batching. We test a small and a large batch size (8 and 128). This operation performs the forward of DLRM on a local batch size of 8 or 128 samples, and synchronizes between batches. It represents the scenario of inference with batching in throughput-oriented applications. (3) Training: in this setting, we evaluate both the forward and backward passes of the embedding layer. We test a local batch size of 8 and 128 to represent small/large batch scenarios. In training, in-network caches are invalidated at the batch boundary.

Inference without batching results: Figure 10a shows the speedup of embedding communication in the forward operation. We show the throughput of the same Aggregation operation under three different setups of switch architecture: cache-only, reduction-only, and cache-and-reduction, normalized to the baseline, which is implemented by the existing *Get* operations.

The results demonstrate the effectiveness of both designs: in a 64-GPU system, cache-only can bring an average of 1.41× speedup, and reduction-only can bring 1.98× speedup, whereas combining both designs gives the highest speedup of 3.12×.

We can observe that the benefit of caching and reduction is related to both the system parallelism and the dataset characteristics, as explained in the next paragraph. Therefore, an architecture that can exploit both input and output reuse in cache and reduction forms is necessary and can be transparently adaptive to systems and datasets.

We can also observe that the benefit of caching and reduction is related to the parallelism in the system, *i.e.*, the number of GPUs. In a 16-GPU system, reduction brings more profit than caching: cache-only brings only 1.03× speedup but reduction brings a significant profit (2.08× between reduction and baseline, 2.25× between cache-and-reduction and cache-only). In contrast, in the 64-GPU and 256-GPU systems, the profits of cache are higher (between cache-only and baseline, 1.41× and 1.25× respectively). We can also observe that the benefit of caching and reduction is dependent on the characteristics of the dataset. For instance, the CTR datasets (Kaggle_ad, Avazu, and Terabyte) are one-hot datasets. One-hot means each input sample only access one embedding entry from each table. For those datasets, reduction brings no profit at all due to a complete lack of output reuse. However, the reduction is crucial for other datasets, especially the fbgemm series. Similarly, caching is more beneficial for CTR datasets than other datasets in 64-GPU system. These observations demonstrate that it is important for our architecture to have both reduction and cache support.

Inference with batching results: Figure 10b and 10c shows the speedup of embedding in inference with batching. When batch size is 8, combining cache and reduction brings 1.73×, 2.05×, and 1.36× speedup for 16-GPU, 64-GPU, and 256-GPU systems respectively. When the batch size is 128, the speedups for the same three systems are 1.96×, 2.29× and 1.73×, respectively. The performance characteristics are similar to the scenario without batching, where the speedups are highly dependent on dataset characteristics.

Training results: Figure 11 shows the speedup of embedding communication in training settings. We only evaluate three datasets which are the largest of their categories. In average, TRACI brings 1.43× speedup for forward and 2.13× speedup for the backward pass. Note that for the CTR dataset avazu, TRACI brings little speedup for the forward (1.00× in average), despite the high speedup for the backward (3.01× in average). This behavior is expected: the forward pass exposes input reuse only because CTR datasets are one-hot, and in training input reuse opportunities are limited because the in-network caches are invalidated between batches. Putting all experiments in this subsection together, we can conclude that TRACI can gracefully adapt to the application and hardware, and exploit reuse opportunities despite that those reuse opportunities are subjective to many factors (dataset, number of GPUs, training or inference, and the batch size).

Evaluation on an alternative topology: Although the multi-GPU NVLink network that TRACI is proposed for uses fat-tree topology, our design is not tied to this topology. Figure 12 shows the speedup of embedding communication in an alternative topology: a system with 64 GPUs connected in a 4×4×4 3d mesh. This topology is used by TPUs [18] which is another mainstream AI

accelerator product. TRACI achieves an average of $1.32\times$ speedup across three datasets and three inference scenarios. The speedup is lower on mesh 3D than the fat-tree topology for the same number of GPUs, which is expected because mesh 3D has longer diameters and smaller fan-out.

6.3 Traffic Reduction

Figure 13 demonstrates the reason for the communication speedup of our architecture. Firstly, the cache and reduction optimizations can reduce the number of response message hops significantly: in a small-scale 16-node system, the average hops of the response message is close to 0, meaning the majority of responses only traverse through 1 switch which is the ideal case. Secondly, the cache and reduction techniques can significantly reduce the workloads of both intra-node and inter-node network links (up to $2.30\times$ and $5.39\times$ respectively). Finally, the optimization significantly reduces the bi-sectional traffic per link, which is average traffic in links that bi-partition all nodes. This clearly reflects why the communication time can be significantly reduced by our optimizations.

6.4 Sensitivity Study

6.4.1 Sensitivity to sizes of the reduction table and the cache. Figure 14 shows the speedup correlated to different sizes of the reduction table and in-switch cache. We conduct the experiment in a simulation setup of 64 GPUs on the representatives of the three dataset series. We observe that the speedup generally increases as the size of the cache and reduction table increases except for the cache speedup on Kaggle_ad. However, the increased area and power overhead is also not negligible. Thus, the hardware overhead and the performance benefit need to be carefully balanced by system designers.

6.4.2 Sensitivity to the system scale. Figure 15 shows the performance benefit of in-switch cache and reduction under different system scales. We use representatives from the three sources of datasets. The other datasets in the same series expose similar characteristics but we omit them in Figure 15 for better visualization. We evaluate the inference without batching Aggregation. For fbgemm_0, we use 10% of all the embedding tables to reduce the simulation time.

Figure 15 shows that the effectiveness of cache optimization is subject to the dataset: for fbgemm_0 the speedup is stable for different number of GPUs, for DBLP it is first stable and then increasing, but for Kaggle_ad it is constantly changing. Our observation is that the cache performance is highly correlated to the dataset but does not expose a trend in the number of GPUs. However, the benefit of the reduction technique, as shown in Figure 15, exposes a clear trend of first increasing and then decreasing. This trend is observed from both fbgemm_0 and DBLP. Note that the reduction-only speedup for Kaggle_ad is always 1 because it is a one-hot dataset with no output reuse.

We further investigate the reason of this trend with the reduction design benefit. Figure 16 shows the reduction table drop-rate and miss-rate of different switches, which explains the reasons. The drop-rate means the percentage of packets being reduced, and the miss-rate means the percentage of packets being bypassed because the reduction table is full. The speedup first increases because the spine switch can reduce the traffic from more leaf

switches. However, because the embedding table is distributed to more GPUs, the reduction opportunity within one node decreases, and the average combination rate of leaf switches decreases. At 128 GPU and 256 GPU cases, the reduction table size is not large enough and some packets are bypassed, losing traffic reduction opportunities. This further limits the obtained speedup. This is the reason behind the trend of reduction table speedup.

6.5 Application Speedup

Figure 17 shows the end-to-end speedups of a DLRM model inference. We use Astra-sim [38] to estimate the percentage of end-to-end latency spent on embedding communication and the rest of the model (e.g., MLP). Then we take the evaluated speedup of embedding communication in Section 6.2 to derive the end-to-end speedup. Note that our design only accelerates embedding communication and does not accelerate the rest of the model. Because the embedding communication is a performance bottleneck in the application, our TRACI design can deliver significant application speedup ($1.32\times$ - $2.68\times$) under various system configurations.

6.6 Hardware Overhead

Table 4: Area overhead of the TRACI modules

	Size	Area (mm^2)	Percentage
In-Switch Cache	2 MB	3.37	1.15%
Reduction Table	2 MB	4.92	1.67%
Total		8.29	2.82%

This subsection elaborates the hardware overhead of TRACI design. In our design, the in-switch cache and the reduction table are both 2MB as shown in Table 4. Each cache line (or RTB entry) are 256B and can store 64 fp32 numbers, and there are 8192 cache lines (or RTB entries). This is the same configuration as used by performance evaluation in Section 6.2-6.5. The major overhead of our switch architecture comes from the reduction table and the in-switch cache. We use Cacti to evaluate the area of the two components in 22nm and scale them to the same process with NVLink4 NVSwitch. The total area of the NVLink4 NVSwitch, as reported in Ishii *et al.* [15], is $294mm^2$ in the TSMC 4N process. The estimated area of TRACI components is shown in Table 4. The cache and reduction table together bring 2.82% extra area to the NVLink Switch.

7 Other Related Work

In-network reduction: There are mainly two types of related work that explore the idea of in-network reduction. The first type targets the All-Reduce collective in deep learning training applications. This type includes designs on both datacenter network switches [9, 10, 28, 29] and inter-GPU memory fabrics [22]. Products like NVIDIA Sharp and NVIDIA Switch Systems also follow this idea to accelerate All-Reduce. A major difference from our work is that none of the prior work is capable of accelerating Aggregation due to its input-dynamic feature. The second type of in-network

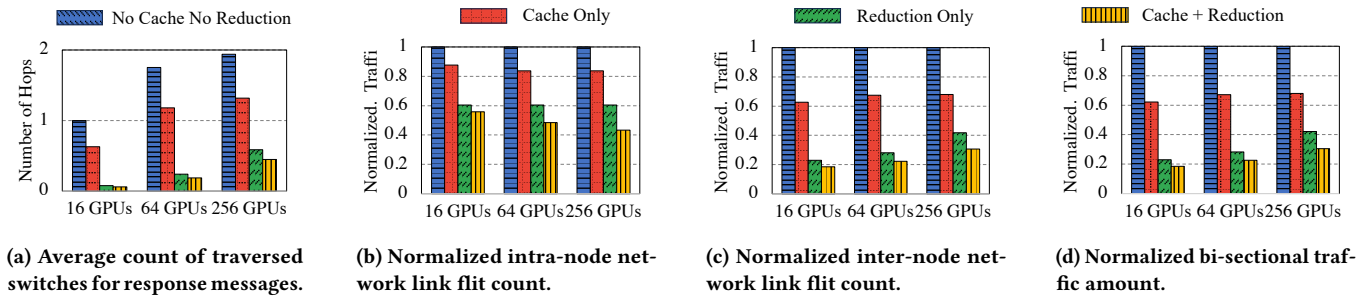


Figure 13: Comparison of the baseline and optimized architectures in terms of the message hop count and traffic.

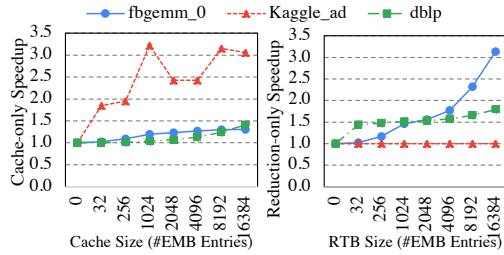


Figure 14: Sensitivity to hardware parameters using 64 GPUs.

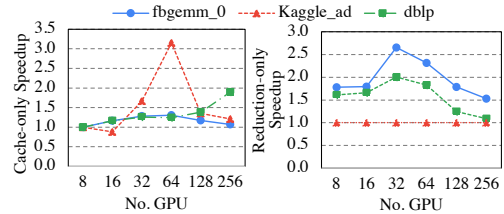


Figure 15: Performance sensitivity to the number of GPUs.

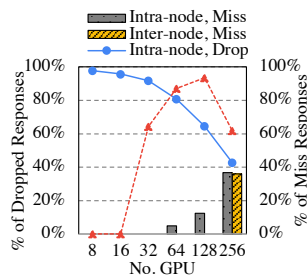


Figure 16: RTB effectiveness for the fbgemm dataset.

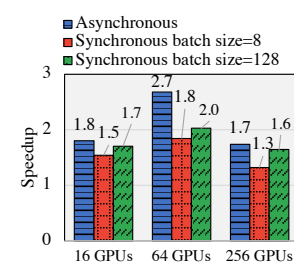


Figure 17: DLRM end-to-end speedup.

reduction architectures target the input-dynamic aggregation collective similar to DLRM embedding, but used in database and map-reduce applications [5, 6, 26, 41]. The major difference between the traditional network and the GPU memory-semantic network that we target is that, communications in traditional network are formed by point-to-point messages but in GPU memory-semantic

network they are request-response transactions. A key component of TRACI is the GetReduce transaction, which extends existing load transactions to support reduction.

In-network caching: In-network caching, with main application in database key-value stores, is widely explored [17, 27, 30, 31, 42]. Our work is different from previous in-network cache studies in the following aspects. Firstly, prior work studies regular network protocols, whereas we study the accelerator fabric with memory operation semantics, leading to different packet granularity, software integration and switch organization. Secondly, we are the first work to make cache and reduction work collaboratively in network.

Comparison to CPU-GPU hybrid systems: Many recent work proposes to use CPU-GPU hybrid systems to efficiently train and deploy DLRMs [4, 12, 25]. Assume CPU-GPU heterogeneous systems uses 1 node of 8 GPUs and infinite backup storage, GPU-only systems use only GPU device memory to fit the DLRM. In terms of cost, because CPU instances and CPU memory are much cheaper than GPU, CPU-GPU heterogeneous system wins. Performance is mainly decided by bandwidth since DLRM application is bottlenecked by Embedding operation. In CPU-GPU systems, CPU memory provides capacity but cannot increase the bandwidth, the bandwidth limit is still the sum of GPU bandwidth in one node. In GPU-only systems the bandwidth is limited by bi-section network bandwidth of the entire system, usually it surpasses the CPU-GPU system at 8 nodes and beyond. In terms of scalability, CPU-GPU systems can scale memory capacity but not performance, while GPU-only system can scale memory capacity and performance simultaneously. In summary, GPU-only system has better performance and scalability than CPU-GPU systems but is more expensive. TRACI enhances the performance of GPU-only systems.

8 Conclusion

To overcome the communication bottleneck in multi-GPU DLRM systems, we propose a network architecture that can dynamically harness the data-dependent input reuse and output reuse opportunities in Aggregation. The proposed solution significantly accelerates the embedding communication in DLRM inference and training.

Acknowledgments

This work is supported by research funding from Samsung Electronics. This work is in part supported by Research Grants Council of HKSAR (No. 16213824).

References

- [1] Bilge Acun, Matthew Murphy, Xiaodong Wang, Jade Nie, Carole-Jean Wu, and Kim Hazelwood. 2021. Understanding training efficiency of deep learning recommendation models at scale. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 802–814.
- [2] Muhammad Adnan, Yassaman Ebrahimzadeh Maboud, Divya Mahajan, and Prashant J Nair. 2021. Accelerating recommendation system training by leveraging popular choices. *Proceedings of the VLDB Endowment* 15, 1 (2021), 127–140.
- [3] Niket Agarwal, Tushar Krishna, Li-Shiuan Peh, and Niraj K Jha. 2009. GARNET: A detailed on-chip network model inside a full-system simulator. In *Performance Analysis of Systems and Software, 2009. ISPASS 2009. IEEE International Symposium on*. IEEE, 33–42.
- [4] Ehsan K Ardestani, Changkyu Kim, Seung Jae Lee, Luoshang Pan, Valmiki Ramersad, Jens Axboe, Banit Agrawal, Fuxun Yu, Ansha Yu, Trung Le, et al. 2021. Supporting Massive DLRM Inference Through Software Defined Memory. *arXiv preprint arXiv:2110.11489* (2021).
- [5] Ge Chen, Gaoxiong Zeng, and Li Chen. 2021. P4com: In-network computation with programmable switches. *arXiv preprint arXiv:2107.13694* (2021).
- [6] Li Chen, Ge Chen, Justinas Lingys, and Kai Chen. 2018. Programmable switch as a parallel computing device. *arXiv preprint arXiv:1803.01491* (2018).
- [7] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishii Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Isipir, et al. 2016. Wide & deep learning for recommender systems. In *Proceedings of the 1st workshop on deep learning for recommender systems*. 7–10.
- [8] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- [9] Daniele De Sensi, Salvatore Di Girolamo, Saleh Ashkboos, Shigang Li, and Torsten Hoeftler. 2021. Flare: Flexible in-network allreduce. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–16.
- [10] Nadeen Gebara, Manya Ghobadi, and Paolo Costa. 2021. In-network aggregation for shared machine learning clusters. *Proceedings of Machine Learning and Systems* 3 (2021), 829–844.
- [11] Carlos A Gomez-Urbe and Neil Hunt. 2015. The netflix recommender system: Algorithms, business value, and innovation. *ACM Transactions on Management Information Systems (TMIS)* 6, 4 (2015), 1–19.
- [12] Huifeng Guo, Wei Guo, Yong Gao, Ruiming Tang, Xiuqiang He, and Wenzhi Liu. 2021. Scalefreectr: Mixcache-based distributed training system for ctr models with huge embedding table. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1269–1278.
- [13] Udit Gupta, Carole-Jean Wu, Xiaodong Wang, Maxim Naumov, Brandon Reagen, David Brooks, Bradford Cottel, Kim Hazelwood, Mark Hempstead, Bill Jia, et al. 2020. The architectural implications of facebook’s dnn-based personalized recommendation. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 488–501.
- [14] Kim Hazelwood, Sarah Bird, David Brooks, Soumith Chintala, Utku Diril, Dmytro Dzhulgakov, Mohamed Fawzy, Bill Jia, Yangqing Jia, Aditya Kalro, et al. 2018. Applied machine learning at facebook: A datacenter infrastructure perspective. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 620–629.
- [15] Alexander Ishii and Ryan Wells. 2022. The Nvlink-Network Switch: Nvidia’s Switch Chip for High Communication-Bandwidth Superpods. In *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE Computer Society, 1–23.
- [16] Biye Jiang, Chao Deng, Huimin Yi, Zelin Hu, Guorui Zhou, Yang Zheng, Sui Huang, Xinyang Guo, Dongyue Wang, Yue Song, Liqin Zhao, Zhi Wang, Peng Sun, Yu Zhang, Di Zhang, Jinhui Li, Jian Xu, Xiaoqiang Zhu, and Kun Gai. 2019. XDL: an industrial deep learning framework for high-dimensional sparse data. In *Proceedings of the 1st International Workshop on Deep Learning Practice for High-Dimensional Sparse Data*. 1–9.
- [17] Xin Jin, Xiaozhou Li, Haoyu Zhang, Robert Soulé, Jeongkeun Lee, Nate Foster, Changhoon Kim, and Ion Stoica. 2017. NetCache: Balancing Key-Value Stores with Fast In-Network Caching. In *Proceedings of the 26th Symposium on Operating Systems Principles (Shanghai, China) (SOSP ’17)*. Association for Computing Machinery, New York, NY, USA, 121–136. doi:10.1145/3132747.3132764
- [18] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, et al. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–14.
- [19] Norman P Jouppi, Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B Jablin, George Kurian, James Laudon, Sheng Li, Peter Ma, Xiaoyu Ma, et al. 2021. Ten lessons from three generations shaped Google’s TPuv4: Industrial product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1–14.
- [20] Hongju Kal, Seokmin Lee, Gun Ko, and Won Woo Ro. 2021. SPACE: locality-aware processing in heterogeneous memory for personalized recommendations. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 679–691.
- [21] Liu Ke, Udit Gupta, Benjamin Youngjae Cho, David Brooks, Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim Hazelwood, Bill Jia, Hsien-Hsin S Lee, et al. 2020. Recnmp: Accelerating personalized recommendation with near-memory processing. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 790–803.
- [22] Benjamin Klenk, Nan Jiang, Greg Thorson, and Larry Dennison. 2020. An in-network architecture for accelerating shared-memory multiprocessor collectives. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 996–1009.
- [23] Youngeun Kwon, Yunjae Lee, and Minsoo Rhu. 2019. Tensordimm: A practical near-memory processing architecture for embeddings and tensor operations in deep learning. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 740–753.
- [24] Youngeun Kwon, Yunjae Lee, and Minsoo Rhu. 2021. Tensor casting: Co-designing algorithm-architecture for personalized recommendation training. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 235–248.
- [25] Youngeun Kwon and Minsoo Rhu. 2022. Training Personalized Recommendation Systems from (GPU) Scratch: Look Forward Not Backwards. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. Association for Computing Machinery, 860–873.
- [26] Alberto Lerner, Rana Hussein, Philippe Cudré-Mauroux, and U eXascale Infolab. 2019. The Case for Network Accelerated Query Processing. In *CIDR*.
- [27] Jialin Li, Jacob Nelson, Ellis Michael, Xin Jin, and Dan RK Ports. 2020. Pegasus: Tolerating Skewed Workloads in Distributed Storage with {In-Network} Coherence Directories. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 387–406.
- [28] Youjie Li, Iou-Jen Liu, Yifan Yuan, Deming Chen, Alexander Schwing, and Jian Huang. 2019. Accelerating distributed reinforcement learning with in-switch computing. In *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 279–291.
- [29] Youjie Li, Jongse Park, Mohammad Alian, Yifan Yuan, Zheng Qu, Peitian Pan, Ren Wang, Alexander Gerhard Schwing, Hadi Esmailzadeh, and Nam Sung Kim. 2018. A Network-Centric Hardware/Algorithm Co-Design to Accelerate Distributed Training of Deep Neural Networks. In *Proceedings of the 51st IEEE/ACM International Symposium on Microarchitecture (MICRO’18)*. Fukuoka City, Japan.
- [30] Ming Luo, Liang Luo, Jacob Nelson, Luis Cez, Arvind Krishnamurthy, and Kishore Atreya. 2017. Incbricks: Toward in-network computation with an in-network cache. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*. 795–809.
- [31] Zaoxing Liu, Zhihao Bai, Zhenming Liu, Xiaozhou Li, Changhoon Kim, Vladimir Braverman, Xin Jin, and Ion Stoica. 2019. {DistCache}: Provable Load Balancing for {Large-Scale} Storage Systems with Distributed Caching. In *17th USENIX Conference on File and Storage Technologies (FAST 19)*. 143–157.
- [32] Meta. 2022. TorchRec. github.com/pytorch/torchrec
- [33] Dheevatsa Mudigere, Yuchen Hao, Jianyu Huang, Zhihao Jia, Andrew Tulloch, Srinivas Sridharan, Xing Liu, Mustafa Ozdal, Jade Nie, Jongsoo Park, Liang Luo, Jie Amy Yang, Leon Gao, Dmytro Ivchenko, Aarti Basant, Yuxi Hu, Jiyan Yang, Ehsan K. Ardestani, Xiaodong Wang, Rakesh Komuravelli, Ching-Hsiang Chu, Serhat Yilmaz, Huayu Li, Jiyuan Qian, Zhuobo Feng, Yinbin Ma, Junjie Yang, Ellie Wen, Hong Li, Lin Yang, Chonglin Sun, Whitney Zhao, Dimitry Melts, Krishna Dhulipala, K. R. Kishore, Tyler Graf, Assaf Eisenman, Kiran Kumar Matam, Adi Gangidi, Guoqiang Jerry Chen, Manoj Krishnan, Avinash Nayak, Krishnakumar Nair, Bharath Muthiah, Mahmoud khorashadi, Pallab Bhattacharya, Petr Lapukhov, Maxim Naumov, Ajit Mathews, Lin Qiao, Mikhail Smelyanskiy, Bill Jia, and Vijay Rao. 2022. Software-hardware co-design for fast and scalable training of deep learning recommendation models. In *ISCA ’22: The 49th Annual International Symposium on Computer Architecture, New York, New York, USA, June 18 - 22, 2022*. ACM, 993–1011.
- [34] Maxim Naumov, John Kim, Dheevatsa Mudigere, Srinivas Sridharan, Xiaodong Wang, Whitney Zhao, Serhat Yilmaz, Changkyu Kim, Hector Yuen, Mustafa Ozdal, et al. 2020. Deep learning training in facebook data centers: Design of scale-up and scale-out systems. *arXiv preprint arXiv:2003.09518* (2020).
- [35] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G Azzolini, et al. 2019. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091* (2019).
- [36] Nvidia. [n. d.]. Nvidia Merlin HugeCTR. <https://developer.nvidia.com/nvidia-merlin/hugctr>
- [37] NVIDIA. 2022. NVIDIA H100 Whitepaper. <https://resources.nvidia.com/en-us-tensor-core>
- [38] Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2020. ASTRA-SIM: Enabling SW/HW Co-Design Exploration for Distributed DL Training Platforms. In *IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2020, Boston, MA, USA, August 22-26, 2020*. IEEE.

- [39] Geet Sethi, Bilge Acun, Niket Agarwal, Christos Kozyrakis, Caroline Trippel, and Carole-Jean Wu. 2022. RecShard: statistical feature-based memory optimization for industry-scale neural recommendation. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 344–358.
- [40] Brent Smith and Greg Linden. 2017. Two decades of recommender systems at Amazon. com. *Ieee internet computing* 21, 3 (2017), 12–18.
- [41] Muhammad Tirmazi, Ran Ben Basat, Jiaqi Gao, and Minlan Yu. 2020. Cheetah: Accelerating database queries with switch pruning. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 2407–2422.
- [42] Qing Wang, Youyou Lu, Erci Xu, Junru Li, Youmin Chen, and Jiwu Shu. 2021. Concordia: Distributed Shared Memory with In-Network Cache Coherence. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. 277–292.
- [43] Weijie Zhao, Jingyuan Zhang, Deping Xie, Yulei Qian, Ronglai Jia, and Ping Li. 2019. Aibox: Ctr prediction model training on a single node. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. 319–328.
- [44] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep interest evolution network for click-through rate prediction. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 5941–5948.